UAV Coverage Path Planning Based on Deep Reinforcement Learning

Yuehai Wang
School of Information, North China University of Technology
Beijing, China
wangyuehai@ncut.edu.cn

Zixin Wang
School of Information, North China University of Technology
Beijing, China
764472833@qq.com

Na Xing
School of Information, North China University of Technology
Beijing, China
xingna@ncut.edu.cn

Shuai Zhao
School of Information, North China University of Technology
Beijing, China
Ld_zhaoshuai@163.com

2023 IEEE 6th International Conference on Computer and Communication Engineering Technology (CCET) | 979-8-3503-4068-6/23/$31.00 ©2023 IEEE | DOI: 10.1109/CCET59170.2023.10335145

***Abstract*—This paper proposes a approach to the problem of low coverage rate in the area coverage path planning of unmanned aerial vehicle (UAV) under the traditional Double Deep Q Network (DDQN). Our approach involves introducing noise parameters into the DDQN and using exploratory noise for deep exploration, which improves the efficiency of exploration. Compared to the traditional DDQN algorithm, our improved method enhances the exploration ability and convergence speed of the trained network, ultimately achieving a higher level of periodic reward. This enables the UAV to better avoid obstacles or no-fly zones in coverage path planning, avoiding it from being stuck or repeating paths in local environments, thereby enabling the UAV to cover more target areas in a shorter number of steps.**

Keywords—Double Deep Q network (DDQN); Exploration strategy; Coverage path planning

I. Introduction

Coverage Path Planning (CPP) [1] for UAV is one of the core areas of research in the field of UAV. UAV are envisioned to be used for a variety of applications, all of which require efficient and safe path planning methods that combine task objectives with navigation constraints, such as flight time and obstacle avoidance. As the name suggests, coverage path planning is related to traditional path planning, which involves finding a path between a start and a target location, but its goal is to cover all points of interest within a region of interest. Generally, the objective of CPP is to cover as much of the target area as possible within given energy or path length constraints while avoiding obstacles or no-fly zones. With the advancement of technology, UAV face increasingly complex and dynamic environments, and traditional path planning can no longer meet the needs of UAV, which has led to the development of algorithms such as ant colony optimization [2] and neural networks [3]. In response to this, researchers have integrated deep learning (DL) with reinforcement learning (RL) [4] to create Deep Reinforcement Learning (DRL) [5], where DL is responsible for utilizing the perception function of neural networks to extract features from unknown environmental states, and RL is responsible for making decisions based on the output of the deep neural network and a certain exploration strategy, thereby achieving the mapping of states to actions and better meeting the mobility needs of UAV.

Mnih et al. [6] combined Convolutional Neural Networks (CNNs) with the Q-learning algorithm in traditional reinforcement learning and proposed a Deep Q-Network (DQN) model to solve decision-making problems based on high-dimensional perception, which was well validated on Atari 2600 games. Van Hasselt et al. [7] proposed a Double Deep Q-Network (DDQN) method that uses different neural network models for action selection and policy evaluation, successfully addressing the problem of value function overestimation. Schaul et al. [8] proposed the Prioritized Experience Replay mechanism, which efficiently utilizes samples to help agents adapt to the environment quickly. Xin et al. [9] first proposed a mobile robot path planning method based on DQN. Wang Z et al. [10] proposed the Dueling DQN model, which decomposes the value function into a state-value function and an advantage function based on the model structure, further improving its performance. DONG Y et al. [11] proposed the Improved Dueling Deep Double Q-Network (IDDDQN) method based on a competitive network structure, making function estimation more accurate and network models more stable. However, using the ϵ--greedy exploration strategy for action selection in the Double Deep Q-Network may not lead to very effective policies, which can affect the performance and reward level of unmanned aerial vehicles.

Therefore, an improved Double Deep Q-Network (DDQN) path planning algorithm for drone coverage is proposed. The method uses a Noisy Network to update the weight parameters of the DDQN network and improve the exploration ability of the drone. The method has the following characteristics: the network model has correlations between state sequences and efficient exploration between state sequences; unlike the traditional ϵ--greedy local shaking strategy, it does not require an additional exploration factor to converge to the optimal solution, nor does it suffer from problems such as requiring a large amount of exploration, low data utilization rate, and long exploration time; the exploration method of Noisy Network is smoother, with finer granularity control over exploration, and adds adaptive noise to decision-making, sampling perturbations

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 08,2026 at 09:42:29 UTC from IEEE Xplore. Restrictions apply.

from the noise distribution, enabling the agent to effectively explore new states.

The structure of this article is as follows: First, it introduces the theory of deep reinforcement learning related to the method of this paper, which will help readers better understand the basic principles and implementation methods of the deep reinforcement learning algorithm in this paper. Secondly, the improvement points of the method in this paper are described, and the network structure is designed. Finally, a simulation experiment scene is designed and compared with the original algorithm, which proves the effectiveness and superiority of the method in this paper.

II. METHODOLOGY

In this section, we will review the theories of reinforcement learning, DQN and DDQN algorithms.

A. Overview of reinforcement learning

Reinforcement learning is a method for an agent to learn the optimal control policy by interacting with the environment through trial and error. It has been widely applied in social sciences, natural sciences, and various engineering fields [12] . The learning process of reinforcement learning is a dynamic and interactive process that is subject to randomness. As shown in Figure 1, in reinforcement learning, the agent makes decisions based on the observed environment (Env) state, obtains the current state S, selects an action A, executes it to obtain the next new state St, and generates an observation reward value R that is fed back to the agent. The agent adjusts itself based on the reward value and selects the next action value based on the current state value, thus continuously iteratively optimizing its decision-making mechanism.

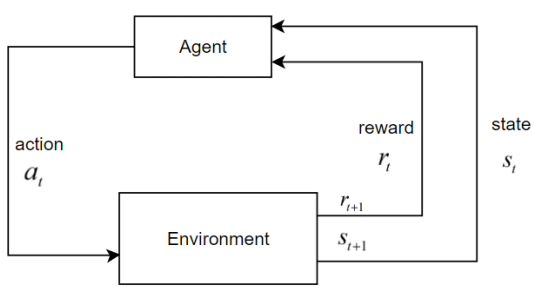


Fig. 1. Schematic diagram of reinforcement learning principle.

B. Deep Network (DQN)

Mnih et al. proposed the DQN algorithm, which combines deep convolutional neural networks with Q-learning algorithms, to some extent addressing the instability problem of algorithms when using non-linear function approximators to represent value functions. The basic framework of the DQN is shown in Figure 2. There are two neural networks with identical structures but different parameters in the deep Q-network, and the DQN algorithm uses two convolutional neural networks, the estimation network and the target network, to represent the state value function and the target value function, respectively. The output of the estimation network is used to evaluate the value function of the current state-action pair, represented by $Q(s,a|\theta t)$. The output of the target network is represented by $Q(s,a|\theta t^{-})$, and the optimization objective of the value function is represented by Y_t approximately (as shown in formula (1)). The target network updates its parameters with the prediction network's parameters every certain period, reducing the correlation between current value $Q(s,a|\theta t)$ and target value $Q(s,a|\theta t^{-})$ to avoid the neural network's parameter training becoming trapped in a local optimal solution. The weight θ of the estimation network is updated in real-time, and after N round of iteration, the weight parameters of the estimation network are assigned to the target network. Initially, $\theta = \theta^{-}$,the estimation network's parameters are set to random values, and the DQN updates the weight values of the network through TD error, as shown in formula (2).

$$Y_t = r + \gamma\, max_{at+1} Q\left(s_{t+1}, a_{t+1} | \theta_t^{-}\right) \quad (1)$$

$$L_t(\theta_t) = E\left(Y_t - Q(s,a|\theta_t)\right)^2 \quad (2)$$

DQN uses an experience replay mechanism to process transition samples obtained during the training process. The Q value calculation formula is shown in formula (3). To converge to the optimal Q value, when exploring the environment, the agent stores the transition samples $e_t = (s_t, a_t, r_t, s_{t+1})$ of the interaction between the drone and the environment in a experience replay unit D at each time step t. During the network training process, a fixed number of transition samples (mini-batch) are randomly sampled from the experience replay unit D each time, and the network parameters are updated using the stochastic gradient descent (SGD) algorithm, as shown in formula (4).

$$Q^{\pi}(s(t),a(t)) = \mathrm{E}_{\pi}[\sum_{k=t}^{T}\gamma^{k-t}R(s(k),a(k),s(k+1))] \quad (3)$$

$$\nabla_{\theta_t} L(\theta_t) = E_{s,a,r,s_{t+1}}[\left(Y_t - Q(s,a|\theta_t)\right)\nabla_{\theta_t} Q(s,a|\theta_t)] \quad (4)$$

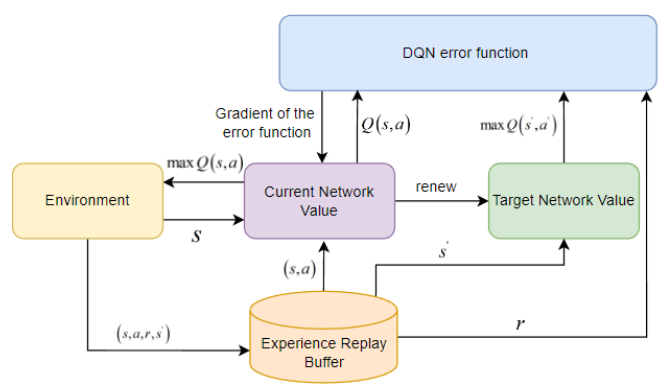


Fig. 2. Basic framework of DQN

C. Double deep Q-network (DDQN)

Hasselt et al. proposed the DDQN (Double Deep Q-Network) algorithm based on the double learning algorithm, which is an improvement on the Q- Learning algorithm. The DDQN network model is shown in Figure 3.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 08,2026 at 09:42:29 UTC from IEEE Xplore. Restrictions apply.

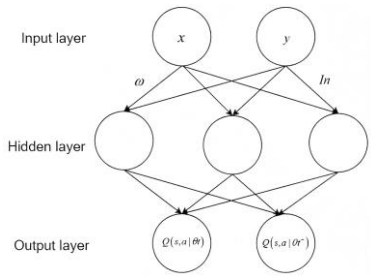


Fig. 3. DDQN model

The main difference between DDQN and DQN is that DDQN uses two neural networks (one for action value evaluation and one for action selection) to solve the problem of overestimating Q-values when selecting the next action in the DQN algorithm. DDQN uses two different sets of parameters and μ^-, where one set of parameters μ is used to select the action of Q value and the other set μ^- is used to evaluate the Q value of the optimal action. Separating the action selection and policy evaluation using two sets of parameters effectively reduces errors, improves algorithm efficiency, and reduces the risk of overestimating Q values. The target value function is shown in formula (5).

$$Y_t^{DDQN} = r + \gamma Q\left(arg\ maxQ\left(s_{t+1}, a_{t+1} \mid \mu_t\right)\mu_t^-\right) \quad (5)$$

III. OUR METHODS

The coverage path planning of the agent is the task of interaction between the agent and the environment. In this paper, a drone coverage path planning method based on DDQN is proposed. First, the target map and the environment map are used as inputs to train the network, and then the optimal action is selected based on the noisy network to reach the next observable observation. The data is then stored in the memory buffer unit D according to the sampling mechanism. This process is iterated continuously until the training is completed.The training model of UAV path planning is shown in Figure 4.

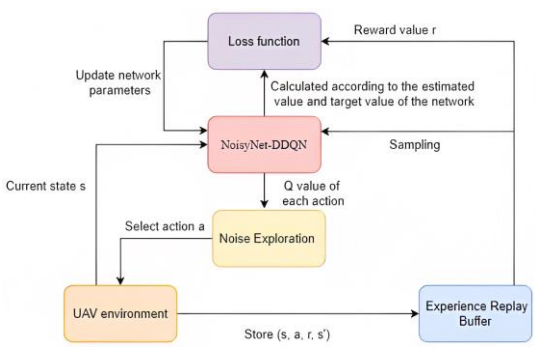


Fig. 4. The training model of UAV coverage path planning

To address the issues mentioned earlier, this paper's algorithm incorporates action space noise into the network structure of the DDQN algorithm based on the Deep Q-Network Based on Action Space Noise proposed by Wu et al. [13]. Gaussian noise distribution is utilized to drive exploration through variance, enabling the agent to discover numerous new states, provide more diverse samples, and offer effective information for decision-making. The network model is shown in Figure 5.

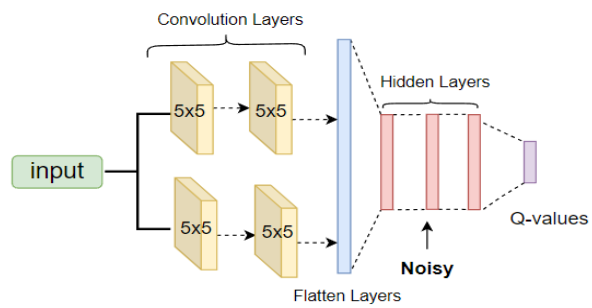


Fig. 5. Network model

The ϵ-greedy policy is one of the most commonly used random policies in traditional reinforcement learning. This policy uses the maximum part of the action-value function for exploitation, while other non-optimal actions are explored with a certain probability. However, this strategy may not achieve optimal exploration because explorations like local jitter strategies cannot reach optimal states. Therefore, to solve this problem, noise parameters can be introduced in the action space to achieve deep exploration by the agent.

After introducing adaptive noise parameters, the network will experience small changes that will persist over multiple time steps in the future and have inherent complexity [14]. These changes lead to changes in the output policy, allowing the policy to explore new states in the next stage. NoisyNet-DDQN uses a greedy policy to randomize the action value function by parameterizing the fully connected layer of the value network as a noise network, and after multiple time steps, extracting the parameters from the noise network parameter distribution. By adding noise to the value function, the final value output can be affected, thereby affecting the choice of action. Therefore, after adding noise to the two network models, the new objective function will be redefined, as shown in formula 6:

$$L_t(\zeta_t) = E[E_{(s,a,r,s_{t+1})}[r + Q\arg\max Q((s_{t+1}, a_{t+1}, \varepsilon_{t+1}; \zeta_{t+1} \mid \mu_t)\mu_t^-) - Q(s_t, a_t, \varepsilon_t; \zeta_t)]^2] \quad (6)$$

Where ε and ε_{t+1} represent noise random variables, and noise parameters are ζ and ζ_{t+1}. When the noise layer is used in place of the linear layer in the network, the parameterized action-value function $Q(s,a;\theta_t)$ can be treated as a random variable, and then optimized using objective $L_t(\zeta_t)$ and noise parameter ζ.

IV. EXPERIMENTS

In this section, we mainly introduce the experimental scenario, environmental parameters and task target definition, and compare the performance indicators of DDQN and NoisyNet-DDQN according to the experimental results.

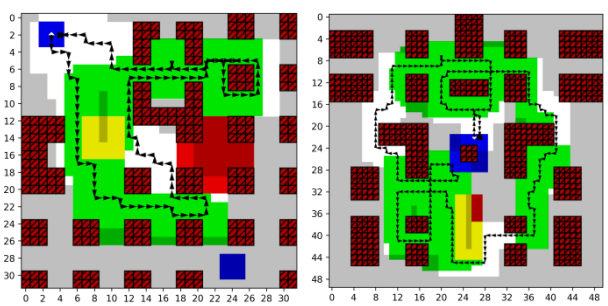
Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 08,2026 at 09:42:29 UTC from IEEE Xplore. Restrictions apply.

A. Experimental Setting

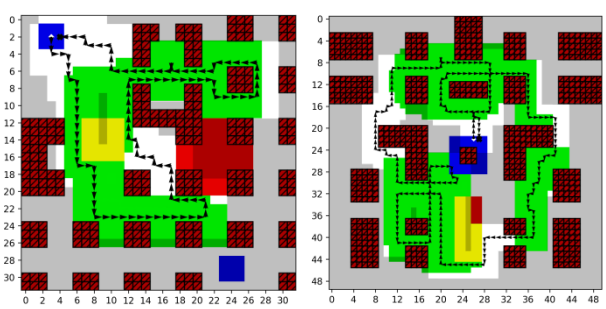
To verify the effectiveness of the proposed algorithm, this study conducted UAV flights in two different grid worlds using compressed global maps and clipped local maps near the UAV agent based on the study by Mirco et al. [15]. Specifically, two scenarios were used in this study, including the "Manhattan32" (Fig. 6a and 6c) and "City50" (Fig. 6b and 6d) scenarios, which have 32 x 32 and 50 x 50 cells, respectively, with a takeoff and landing area located at the upper left and lower right corners ("Manhattan32") or around the center building ("City50"). The cell size in each scenario is 10m x 10m, and the legend of each block is detailed in Table 1.

TABLE I. MAP RELATED INFORMATION REPRESENTATION

Symbol	Description
	Regulatory no-fly zone (NFZ)
	Start and landing zone
	Buildings blocking wireless links and FoV
	Remaining target zone (yellow also NFZ)
	Not covered and covered



(a)Movement 124/150, CR=0.94 (b) Movement 234/250, CR=0.94



(c) Movement 116/150, CR=0.95 (d) Movement 234/250, CR=0.95

Fig. 6. Example trajectories from the Monte Carlo simulations for DDQN (a)+(b) and NoisyNet-DDQN (c)+(d) on 32 × 32 Manhattan map (a)+(c) and 50 × 50 Urban map (b)+(d).

B. Environment and parameter settings

In a square grid scenario with size $M \times M \in N^2$, where N is the set of natural numbers, a cell size of c is set. The environment includes designated takeoff and landing locations, no-fly zones (NFZs), and obstacles. The map can be described using a tensor $M \in B^{M \times M \times 3}$, where $B = \{0,1\}$ describes the takeoff and landing areas on map layer 1, NFZs and obstacles on map layer 2, and only obstacles on map layer 3.

The UAV moves in this environment at a constant altitude h, occupying a single cell of the environment, and its position is determined by $p(t) \in N^2$. Its movement is restricted by obstacles and NFZs. Additionally, to complete its mission, the UAV must start and end the task in the takeoff and landing area within the maximum flight time determined by its initial fuel. The initial fuel of the UAV $b(t)$ is $b_0 \in N$, which decreases by 1 for each action step.

C. Task Definition

In the coverage path planning task, the drone needs to fly over or near a designated target area to ensure that the area is within the camera's field of view below the drone. We used a camera with a viewing angle of 90 degrees, and the drone's fixed flying height is 25 meters. Therefore, in the absence of obstacles, the drone can cover 5x5 cells simultaneously. The target area is formed by randomly sampling different sizes and types of geometric shapes and superimposing them to form a partially connected target area. In this task, we use two performance metrics to evaluate the algorithm's performance. The first metric is the coverage rate (CR), which is the ratio of the number of target cells covered by the drone when the task is completed to the initial number of target cells. The second metric is the coverage rate and landing rate (CRAL). When the drone successfully lands, CRAL is equal to CR; when the drone fails to land, CRAL is equal to 0. Since we have normalized the performance metrics, they can be compared in different scenarios with randomly generated target areas to evaluate the algorithm's performance.

D. Experimental Results

For the "Manhattan32" scenario, the agent was set to complete the task between 50-150 steps, and for the "City50" scenario, the task was set to be completed between 150-250 steps. Figures 6a and 6b show the training of the agent for target area coverage under the DDQN algorithm, with some target areas remaining uncovered due to the ignored small areas that require detours. From Figures 6c and 6d, it can be seen that the coverage path of the agent under the NoisyNet-DDQN network has been optimized, and most of the target areas have been covered, even in the no-fly zone.

The agents in all four scenarios have been trained for 2 million steps. When analyzing the performance of the four in different scenarios or algorithms, as shown in Table 2, it can be seen that the agent indicators in NoisyNet-DDQN include Landed, CR and CRAL are higher than DDQN.

TABLE II. COMPARISON OF DDQN AND NOISYNET-DDQN PERFORMANCE INDICATORS.

Algorithm	NoisyNet-DDQN		DDQN	
Metric	Manhattan32	Urban50	Manhattan32	Urban50
Landed	98.90%	98.60%	98.50%	98.10%
CR	71.82%	82.37%	71.00%	81.50%
CRAL	71.03%	81.01%	70.30%	80.10%

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 08,2026 at 09:42:29 UTC from IEEE Xplore. Restrictions apply.

V. CONCLUSION

In this paper, we proposed an improved DDQN algorithm by introducing noise to the DDQN, which can enhance the exploration speed of the agent in the environment, avoiding the agent getting stuck in local environments or repeating paths. With this, the agent can find the optimal coverage path more quickly and cover more target areas. In summary, the NoisyNet-DDQN method can obtain greater cumulative reward values with fewer iterations, allowing the agent to achieve higher coverage rates in the shortest time possible and in the smallest possible number of steps.. This improvement provides a more efficient solution for unmanned aerial vehicle path planning problems, allowing the agent to complete tasks faster and achieve better results. For the purposes of this paper, only coverage experiments conducted independently by a single UAV were considered, after which multiple UAVs could be carried out to jointly perform coverage path planning studies.

REFERENCES

[1] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, and M. Caccamo, "UAV coverage path planning under varying power constraints using deep reinforcement learning," in IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.

[2] Reshamwala A，Vinchurkar D P.Robot path planning using an ant colony optimization approach：A survey[J].International Journal of Advanced Research in Artificial Intelligence，2013，2（3）：65-71.

[3] Juang Chia-Feng, Yeh Yen-Ting, Chia-Feng Juang, et al. Multiobjective Evolution of Biped Robot Gaits Using Advanced Continuous Ant-Colony Optimized Recurrent Neural Networks[J].IEEE Transactions on Cyberneti cs,2018,48(6):1910-1922.

[4] Sutton R S，Barto A G.Reinforcement learning：An introduction[M].3rd ed.Cambridge，UK：MIT Press，2018：3-11.

[5] Liu Q, Zhai J, Zhang Z C, et al. A survey on deep reinforcement learning[J]. Chinese Journal of Computers, 2018, 41(1):1-27.

[6] Mnih V，Kavukcuoglu K，Silver D，et al. Human-level control through deep reinforcement learning［J］.Nature，2015，518（7540）：529-533.

[7] Hasselt H，Guez A，Silver D.Deep reinforcement learning with double q-Learning[C]//Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence，Phoenix，Arizona USA，February 12-13，2016：2094-2100.

[8] Schaul T，Quan J，Antonoglou I，et al. Prioritized experience replay［C］.Proceedings of the International Conference on Learning Representations，San Juan，Puerto Rico，2016：53-73.

[9] Xin J，Zhao H，Liu D，et al.Application of deep reinforcement learning in mobile robot path planning[C]// Proceedings of Chinese Automation Congress（CAC），Jinan，China，October 20-22，2017：7112-7116.

[10] Wang Z Y，Schaul T，Hessel M，et al. Dueling network architectures for deep reinforcement learning[C] .Proc of the 33th International Conference on Machine Learning，2016：1995-2003.

[11] DONG Yao, GE Yingying, GUO Hongyong, et al. Path planning for mobile robot based on deep reinforcement learning. Computer Engineering and Applications, 2019, 55（13）：15-19.

[12] Sutton R S, Barto A G. Reinforcement learning: An introduction[M]. MIT press, 2018

[13] Wu Xiaming, Li Mingqiu, Chen Enzhi, Wang Chunyang. Deep Q Network Learning Based on Action Space Noise [J]. Journal of Changchun University of Science and Technology (Natural Science Edition), 2020, 43(04): 85-91.

[14] Hou Y，Liu L，Wei Q，et al. A novel DDPG method with prioritized experience replay ［C］. 2017IEEE International Conference on Systems，Man，and Cybernetics（SMC）. IEEE，2017：316-321.

[15] M. Theile, H. Bayerlein, R. Nai, D. Gesbert and M. Caccamo, "UAV Path Planning using Global and Local Map Information with Deep Reinforcement Learning," 2021 20th International Conference on Advanced Robotics (ICAR), Ljubljana, Slovenia, 2021, pp. 539-546, doi: 10.1109/ICAR53236.2021.9659413.

Authorized licensed use limited to: WUHAN UNIVERSITY OF TECHNOLOGY. Downloaded on March 08,2026 at 09:42:29 UTC from IEEE Xplore. Restrictions apply.